FINAL PROJECT DOCUMENTATION

Software Requirements Specification (SRS)
Software Architecture & Design (SAD)
Software Test Plan (STP)

Document Control

Item	Details
Project	StudentRMS – Student Record Management System
Project ID	SRMS-2026-04
Team	Team 4
Stakeholder	Department Academic Office (record-keeping staff)
Document set	SRS + Software Architecture & Design + Software Test Plan

Table of Contents

Document Control 1
Table of Contents 1
1. Project Title and Group 4
2. Team Profile and Product Ownership 4
DELIVERABLE 2 – REQUIREMENTS 5
2.1 SRS – Overview / Required Structure 6
2.2 Purpose 6
2.3 Scope 6
2.4 User Classes 7
2.5 Functional Requirements 8
2.6 Non-Functional Requirements 8
2.7 Security Requirements 9
2.8 Business Rules / Domain Rules 9
2.9 Use Cases 9
2.10 Validation Specification – Initial Planning 10
2.11 Project Plan 10
2.12 Tool Stack 11
Appendix A: Glossary 11
Appendix B: Field Layouts (struct Student / credentials) 11
Appendix C: Requirement Traceability Matrix (Initial) 12
Revision History 13
Approvals 13
1. Introduction 13
1.1 Purpose 13

1.2 Scope 13
1.3 Audience 13
1.4 Definitions 13
2. Document Overview 13
2.1 How to use this document 13
2.2 Related Documents 13
3. Architecture 13
3.1 Goals & Constraints 13
3.2 Stakeholders & Concerns 13
3.3 Component (UML) Diagram 13
3.4 Component Descriptions 14
3.5 Chosen Architecture Pattern and Rationale 14
3.6 Technology Stack & Data Stores 14
3.7 Risks & Mitigations 14
3.8 Traceability to Requirements 14
3.9 Security Architecture 14
4. Design 14
4.1 Design Overview 14
4.2 UML Sequence Diagrams 14
4.3 API / Interface Design 14
4.4 Error Handling, Logging & Monitoring 14
4.5 UX Design 14
4.6 Open Issues & Next Steps 15
5. Appendices 15
5.1 Data / Field Layout 19
5.2 Glossary 19
5.3 References 19
1. Introduction 20
Purpose 20
Scope 20
2. Test Items 20
3. Features to be Tested 20
4. Features Not to be Tested 20
5. Test Approach / Strategy 20
Levels 20
Types 20
5.1 Security Validation 21

6. Test Environment 21
7. Test Schedule 21
8. Test Deliverables 21
9. Roles and Responsibilities 21
10. Risks and Mitigation 21
11. Assumptions & Dependencies 21
12. Suspension & Resumption Criteria 21
13. Test Case Management & Traceability 21
14. Test Metrics & Reporting 21
15. Approvals 21
Submission Check 22

PART I – SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Source: supplied StudentRMS SRS document. The requirements content is retained while the presentation has been reformatted into editable headings, tables and lists.

1. Project Title and Group

Field	Entry
Project Title	StudentRMS – Student Record Management System
Group Number	Team 4
Project ID	SRMS-2026-04
Technology	C C++ (console application, file-based persistence)
Complexity	Easy
Customer Stakeholder	Department Academic Office (record-keeping staff)

2. Team Profile and Product Ownership

To align with the requirement that each member must own at least one complete functional feature, the four functional areas of StudentRMS are split equally below. Each member owns one feature end-to-end and is individually responsible for the matching sections of both the Synopsis and the SRS.

Member	SRN	Owned Functional Feature	Synopsis Deliverables	SRS Deliverables
Member 1 — Samiksha Jayanth		Administrator Authentication & Session Management	Synopsis Deliverables: ● Problem Statement & Objectives for Authentication ● Executive Summary / Abstract	SRS Deliverables: ● Section 2.2: Purpose (Introduction) ● Section 2.4: User Classes & Characteristics ● Section 2.7: Security Requirements (SEC-001, SEC-002, SEC-005) ● FRs: FR-001 (Login/ Authentication) ● Use Cases: UC-01 (Administrator Login)
Member 2 — Rohan M	PES1UG24CS381	Record Search, View & Listing	Synopsis Deliverables: ● Problem Statement & Objectives for Search/Retrieval ● High-Level Scope & Project Boundaries	SRS Deliverables: ● Section 2.3: Scope (In Scope Out of Scope) ● Section 2.6: NFR-001 (Performance) ● FRs: FR-004 (Search), FR-007 (List & Sort) ● Use Cases: UC-03 (Search/ View Record), UC-06 (List

Member 3 — Saksham Bajpayi	PES1UG24CS404	Record Lifecycle Management (Add, Update, Delete)	Synopsis Deliverables: ● Problem Statement & Objectives for Record Management ● Key Deliverables & Expected Outcomes	SRS Deliverables: ● Section 2.5: Functional Requirements (detailed FRs) ● Section 2.6: NFR-002 (Reliability) ● FRs: FR-002 (Add), FR-003 (Duplicate Check), FR-005 (Update), FR-006 (Delete), FR-012 (Validation) ● Use Cases: UC-02 (Add Record), UC-04 (Update Record), UC-05 (Delete
Member 4 — Riddhi Mundada		Reporting, CGPA Computation & Data Persistence	Synopsis Deliverables: ● Problem Statement & Objectives for Reporting/ Persistence ● Target Audience & System Constraints	SRS Deliverables: ● Section 2.2: Design & Implementation Constraints (file-based storage) ● Section 2.12: Tool Stack ● FRs: FR-008 (CGPA Calc), FR-009 (Rank List), FR-010 (Persistence), FR-011 (Export) ● Use Cases: UC-07 (Generate Rank List / Report)

Important: UI design, file-handling support and unit testing are supporting responsibilities, not substitutes for owning a functional feature, per the project guidelines.

DELIVERABLE 2 – REQUIREMENTS

This package contains the Software Requirements Specification (SRS), initial validation planning and the project plan, organized into the same structure as the university-supplied SRS template: introduction, overall description, interfaces, analysis models, system features, non-functional requirements, business rules, glossary, field layouts and a requirement traceability matrix.

2.1 SRS – Overview / Required Structure

Section	What This Package Covers	StudentRMS Example
1. Introduction	Purpose, audience, scope, references	Why the system exists and who reads the SRS
2. Overall Description	Perspective, functions, users, environment, constraints, assumptions	Administrator/console environment, single-user
3. External Interfaces	UI, software and communication interfaces	Console menu, local file I/O
4. Analysis Models	Use case and other relevant models	Use case diagram + record data model
5. System Features	Feature description, stimulus/response, detailed FRs	Add/Search/Update/Delete Record, Report
6. Nonfunctional Requirements	Performance, safety, security, quality attributes, business rules	Response time, authentication, reliability
7. Other Requirements	Database/legal/localization/reuse/etc.	File-format and data-retention requirements
Appendix A	Glossary	CGPA, Record, Administrator
Appendix B	Field layouts/report requirements	struct Student fields
Appendix C	Requirement Traceability Matrix	REQ → architecture → design → code → tests

2.2 Purpose

The purpose of StudentRMS is to provide a console-based platform for an academic-office administrator to add, search, update, delete, sort and report on student records, with automatic percentage/CGPA computation and persistent file-based storage.

2.3 Scope

In Scope	Out of Scope
Administrator authentication	Multi-user role hierarchy (only one administrator role)
Add search update delete student records	Student self-service portal GUI
Automatic percentage CGPA calculation	Networked multi-client access
Sorting and rank-list generation	Integration with a university ERP system
File-based persistent storage	Relational database (DBMS) backend
Text-file report export	Web or mobile interface

2.4 User Classes

User Class	Privileges	Characteristics
Administrator	Full CRUD on student records, generate reports	Single trusted user; comfortable with a text- menu interface

ID	Requirement	Priority	Verification
FR-001	The system shall require an administrator to log in with a username and password before any record-management function is accessible.	High	Functional test
FR-002	The system shall allow an authenticated administrator to add a new student record containing roll number, name, department, semester and subject-wise marks.	High	Functional test
FR-003	The system shall reject an add-record request if the roll number already exists in the data file.	High	Negative test
FR-004	The system shall allow the administrator to search for a student record by roll number or by name.	High	Functional test
FR-005	The system shall allow the administrator to update the fields of an existing student record identified by roll number.	High	Functional test
FR-006	The system shall allow the administrator to delete an existing student record by roll number, after an explicit confirmation step.	High	Functional test
FR-007	The system shall display all student records sorted by roll number, name, or CGPA, as selected by the administrator.	Medium	System test
FR-008	The system shall automatically compute each student's percentage and CGPA from the stored subject marks.	High	Unit test
FR-009	The system shall generate a class rank list ordered by CGPA in descending order.	Medium	System test

FR-010	The system shall persist all student records to a data file such that records are retained after the program exits and are reloaded on the next run.	High	Integration test
FR-011	The system shall allow the administrator to export the current record list / rank report to a text file.	Medium	Functional test
FR-012	The system shall validate that mandatory fields are non-empty and that marks entered are within the range 0–100 before saving a record.	High	Negative + positive test

2.5 Functional Requirements

Requirement-writing rule: each requirement is uniquely identified and is concise, complete, unambiguous, verifiable and necessary, per the supplied SRS template.

2.6 Non-Functional Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	The system shall return the result of a search/list operation for up to 1000 records within 2 seconds on the reference test machine.	Performance test
NFR-002	Reliability	The system shall not corrupt or lose existing records if the program terminates unexpectedly during a write operation.	Fault-injection test
NFR-003	Usability	An administrator shall be able to complete any core operation (add/ search/update/delete) within 3 menu selections from the main menu.	Usability walkthrough
NFR-004	Maintainability	Business logic shall be separated into per-module source/header files with no function exceeding ~60	Code review
NFR-005	Testability	Each core module shall expose functions that can be exercised by a unit-test harness without requiring the interactive console menu.	Unit/integration test
NFR-006	Portability	The system shall compile and run without modification on both Linux (gcc) and Windows (MinGW) using only the C standard library.	Build verification

2.7 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	The administrator password shall not be stored or compared as plaintext embedded in source code; it shall be read from a separate, restricted-access credentials file.	Code/configuration review
SEC-002	No record-management function shall be reachable from the console menu prior to successful administrator authentication.	Authorization test
SEC-003	All user-supplied input (roll number, name, marks) shall be length-checked and type-validated before use, to prevent buffer overflows in fixed-size C buffers.	Negative/security test
SEC-004	File read/write operations shall check return values and handle I/O errors gracefully without crashing or leaving the data file in a partially written state.	Fault-injection test
SEC-005	Failed login attempts shall be logged with a timestamp to a local log file.	System test

2.8 Business Rules / Domain Rules

- Only an authenticated administrator may add, update, or delete a student record.
- A roll number must be unique across all records.
- Marks for any subject must lie between 0 and 100 inclusive.
- Percentage and CGPA are always system-computed, never directly entered by the administrator.
- A record deletion requires explicit confirmation and cannot be undone.

2.9 Use Cases

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Administrator Login	Administrator	Application launched	Authenticated session started
UC-02	Add Student Record	Administrator	Logged in	New record saved to file
UC-03	Search View Record	Administrator	Logged in; records exist	Matching record displayed
UC-04	Update Student Record	Administrator	Record exists	Record updated in file
UC-05	Delete Student Record	Administrator	Record exists	Record removed from file
UC-06	List & Sort Records	Administrator	Logged in	Sorted list displayed
UC-07	Generate Rank List Report	Administrator	Records exist	Report displayed and/or exported

2.10 Validation Specification – Initial Planning

At the requirements stage, the validation specification is planning-focused. This stage is not the final test report; detailed test cases and pass/fail evidence are produced in the Implementation & Testing and Final Report deliverables.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests on each menu operation	Add/search/update/delete a record with valid and invalid data
Security	Authentication bypass attempts; buffer-overflow/oversized-input tests	Attempt record operations without login; enter an over-length name
Performance	Timed list/search operation on a data file with ~1000 records	Measure response time against NFR-001
Usability	Task-based walkthrough with a first-time user	New user adds and retrieves a record unaided
Reliability	Interrupt the program mid-write and restart	Verify data file is not corrupted (NFR-002)
Unit	Automated unit tests + coverage on core functions	CGPA calculation, sort routine, validation functions

2.11 Project Plan

The project-plan guidance asks the team to identify and justify the lifecycle, identify tools, create a detailed WBS, and estimate effort in person-days.

Work Package	Tasks	Owner	Estimated Effort (person-days)	Output
WP1	Requirements + use cases	All	3	SRS
WP2	Architecture & module-level design	All	2	Design document
WP3	Admin authentication & session management	M1 (Samiksha)	4	Working feature
WP4	Search, view & listing module	M2 (Rohan)	4	Working feature
WP5	Record lifecycle: add update delete	M3 (Saksham)	6	Working feature
WP6	CGPA computation, ranking, reporting & persistence	M4 (Riddhi)	6	Working feature
WP7	Testing + CI/CD + security validation	All	4	Test/validation evidence
WP8	Final validation + report	All	3	Final report

Total estimated effort: 32 person-days across a 6-sprint, ~6-week schedule, tracked on the GitHub Projects board with weekly evaluation checkpoints.

2.12 Tool Stack

Activity	Recommended/Example Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects Jira	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Pipeline run logs
Build system	GCC Makefile	Build logs
Testing	Custom assert-based harness Unity	Test results
Static analysis	cppcheck	Analysis report
Design	Draw.io PlantUML	UML diagrams

Appendix A: Glossary

Term	Definition
CGPA	Cumulative Grade Point Average, computed from subject marks.
Record	A single student's stored data (roll number, name, marks, etc.).
Administrator	The single authenticated user permitted to manage records.
Data file	The local file used for persistent storage of all records.

Appendix B: Field Layouts (struct Student / credentials)

Field	Length	Data Type	Description	Mandatory
roll_no	10	char[]	Unique student roll number	Y
name	50	char[]	Student full name	Y
department	20	char[]	Department branch code	Y
semester	2	int	Current semester (1–8)	Y
subject_marks[]	6 x float	float[]	Marks per subject (0–100)	Y
percentage	-	float (computed)	Derived percentage	N (auto)
cgpa	-	float (computed)	Derived CGPA	N (auto)
admin_username	20	char[]	Administrator login ID	Y
admin_password_hash	64	char[]	Stored credential (not plaintext)	Y

Appendix C: Requirement Traceability Matrix (Initial)

Architecture, design, code-file and test-case references will be filled in as the Design and Implementation deliverables are completed; requirement IDs and descriptions are fixed at this stage.

Sl. No	Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
1	FR-001	Administrator login before access					
2	FR-002	Add student record					
3	FR-004	Search record by roll no./name					
4	FR-005	Update student record					
5	FR-006	Delete student record					
6	FR-008	Automatic CGPA computation					
7	FR-010	Persist records to file					
8	SEC-003	Input length/type validation					

PART II – SOFTWARE ARCHITECTURE AND DESIGN (SAD)

The StudentRMS Software Architecture and Design Specification is included below with its tables and UML/ architecture figures preserved as editable Word content.

Revision History
Approvals
1. Introduction

1.1 Purpose

This document specifies the architecture and design of StudentRMS, translating the approved requirements into modules, interfaces, UML views, security controls and validation points.

1.2 Scope

StudentRMS supports administrator authentication, student record add/search/update/delete, listing and sorting, percentage/CGPA computation, rank-list/report generation, text-file export and persistent local file storage. Web/mobile access, multi-user roles, relational databases and ERP integration are out of scope.

1.3 Audience

Developers, QA engineers, course evaluators, stakeholders and maintainers.

1.4 Definitions

StudentRMS, CGPA, FR, NFR, RTM, SEC, Repository and Validation.

2. Document Overview

2.1 How to use this document

Use with the StudentRMS SRS and STP. The SRS defines requirements, the SAD defines implementation structure and the STP defines verification.

2.2 Related Documents

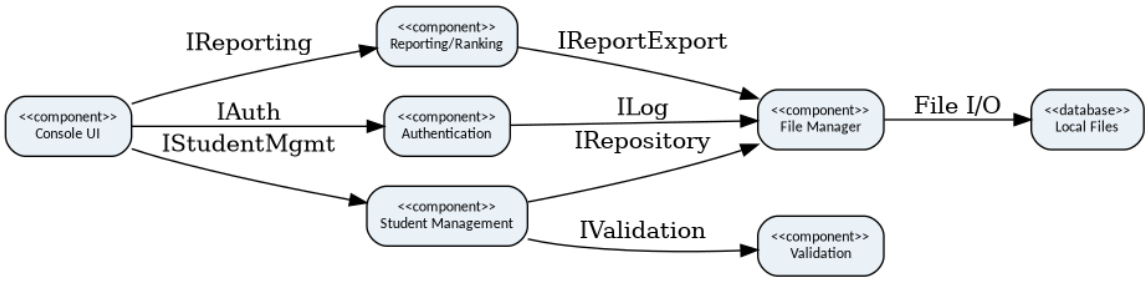
StudentRMS SRS v1.0; StudentRMS STP v1.0; RTM; project source code and validation specification.

3. Architecture

3.1 Goals & Constraints

3.2 Stakeholders & Concerns

3.3 Component (UML) Diagram



3.4 Component Descriptions

3.5 Chosen Architecture Pattern and Rationale

Layered modular architecture is appropriate for this small console application with local file persistence. It separates concerns and supports unit testing without introducing the deployment and network complexity of microservices.

3.6 Technology Stack & Data Stores

standard library; console/terminal interface; local files: students.dat, , login.log and reports.txt.

3.7 Risks & Mitigations

3.8 Traceability to Requirements

3.9 Security Architecture

Security controls enforce administrator authentication, separate credential storage, input validation, safe file handling and timestamped failed-login logging. Tests cover authorization, credential storage, malformed input, file failures and audit logging.

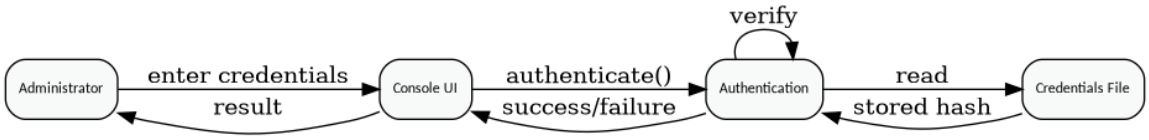
4. Design

4.1 Design Overview

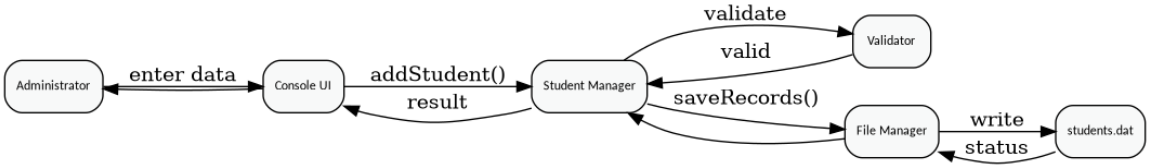
Console UI calls controller/module functions; business rules are enforced by validation; File Manager isolates persistence. Percentage and CGPA are derived from subject marks.

4.2 UML Sequence Diagrams

Sequence 1 – Administrator Login



Sequence 2 – Add Student Record



4.3 API / Interface Design

4.4 Error Handling, Logging & Monitoring

Functions return defined status values. User errors are clear without exposing sensitive data. File return values are checked. Failed login attempts are timestamped. Monitor I/O failures, authentication failures, duplicate attempts and export failures.

4.5 UX Design

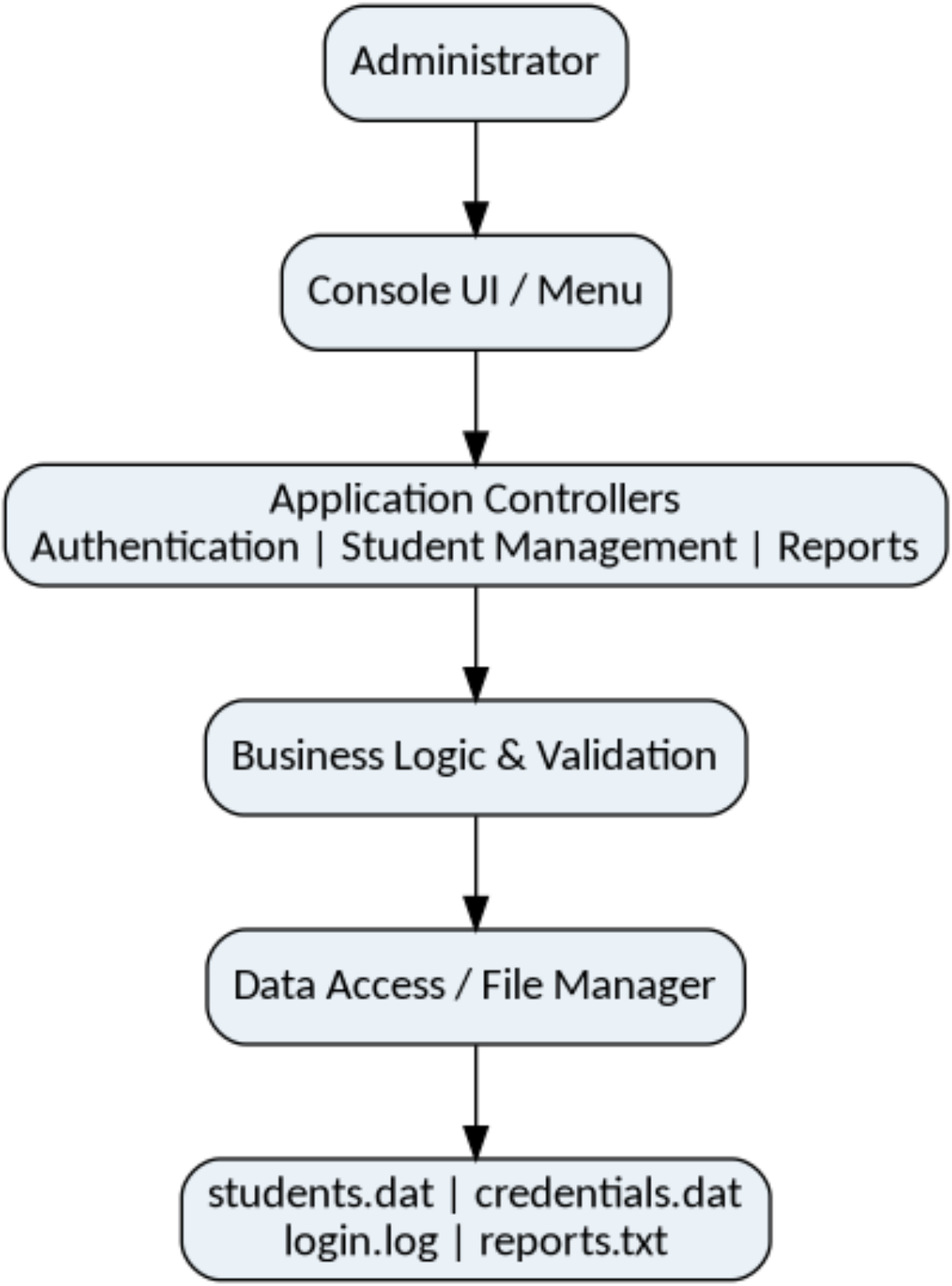
The console provides concise menus/prompts. Destructive deletion requires explicit confirmation. Validation errors provide corrective guidance.

4.6 Open Issues & Next Steps

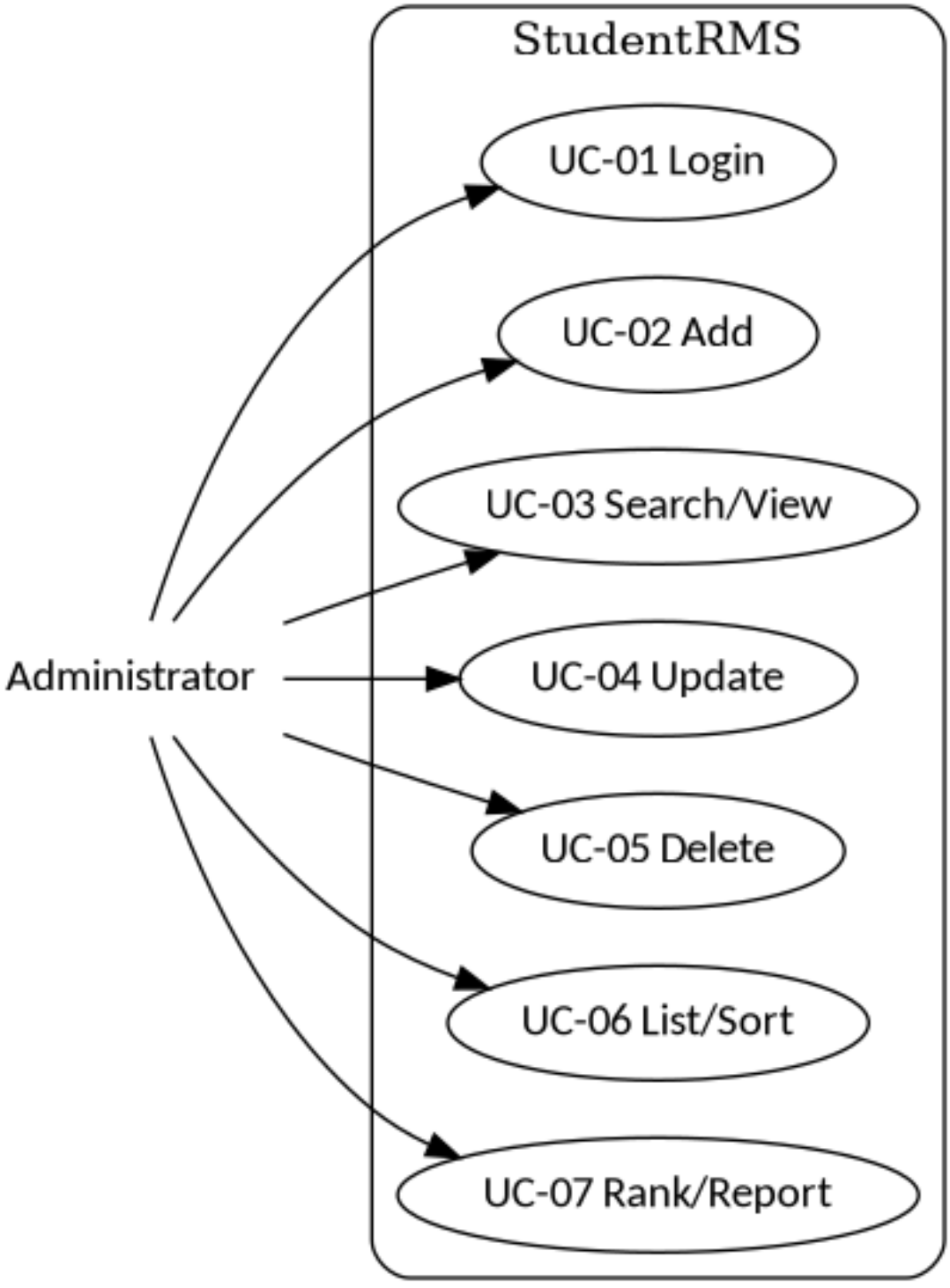
Possible future extensions include a GUI/web interface, multi-user roles, database persistence and ERP integration; these are outside the current scope.

5. Appendices

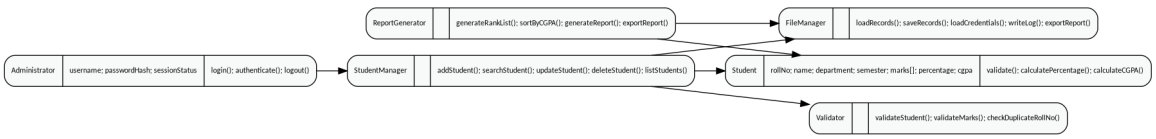
A. High-Level Architecture



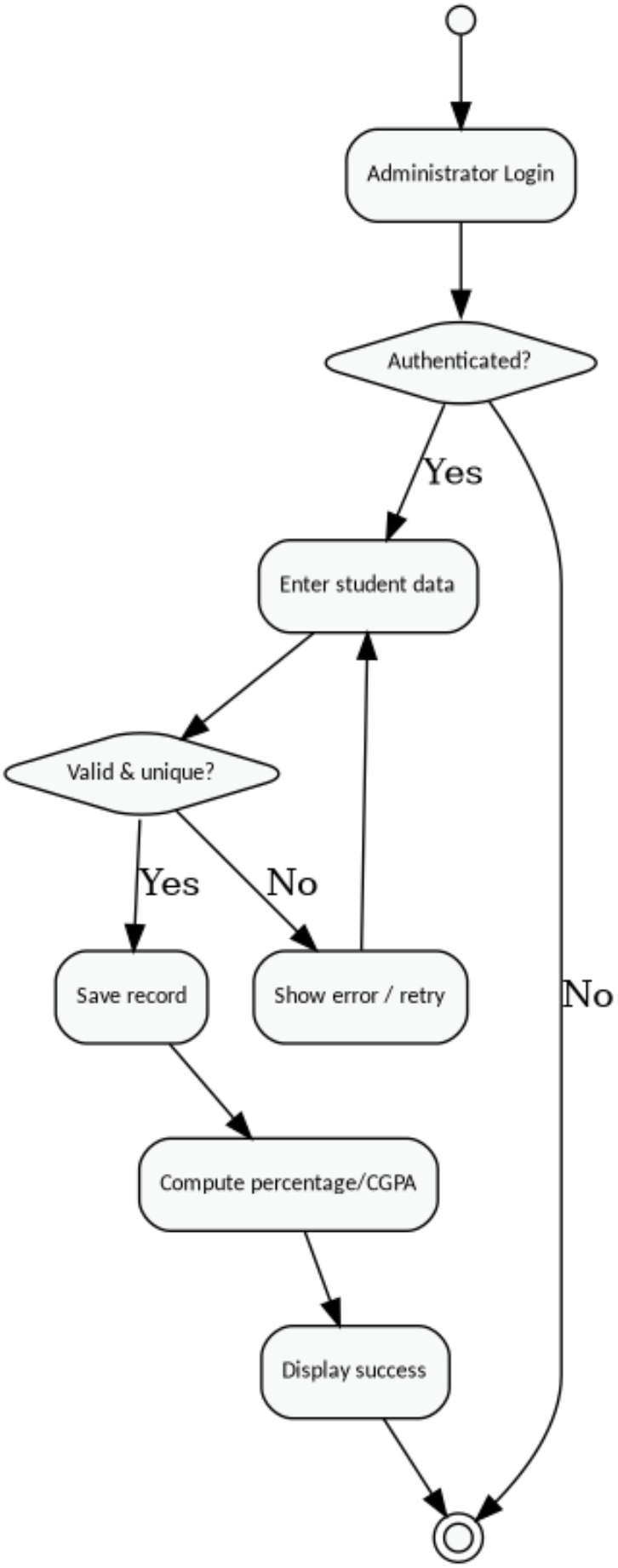
B. Use-Case Diagram



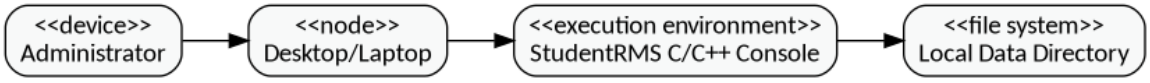
C. Class Diagram



D. Activity Diagram – Add Record


Administrator Login
Authenticated?
Yes
No
Enter student data
Valid & unique?
Yes
No
Save record
Show error / retry
Compute percentage/CGPA
Display success

E. Deployment Diagram



5.1 Data / Field Layout

5.2 Glossary

Administrator; Student Record; CGPA; Repository; Validation; RTM; NFR; SEC.

5.3 References

StudentRMS SRS v1.0; StudentRMS Deliverable 3 Design; supplied SAD template.

PART III – SOFTWARE TEST PLAN (STP)

The StudentRMS Software Test Plan is included below with its test strategy, test cases, schedule, risks and responsibilities.

1. Introduction

Purpose

This plan defines objectives, scope, strategy, resources, test cases, security validation, risks and traceability for StudentRMS.

Scope

Authentication, CRUD, validation, search/list/sort, CGPA, ranking, reporting/export, persistence, reliability, performance, portability and security. Web/mobile UI, ERP, relational DB and external integrations are excluded.

2. Test Items

- Authentication
- Student Management
- Validation
- Search and Listing
- CGPA and Ranking
- Reporting/Export
- File Manager/Persistence
- Console UI

3. Features to be Tested

4. Features Not to be Tested

- University ERP/external integration
- Web/mobile GUI
- Relational database behavior
- OS/compiler internals

5. Test Approach / Strategy

Levels

- Unit: validation, authentication helpers, CGPA, ranking and file utilities.
- Integration: Student Management + Validation + File Manager; Reporting + Repository.
- System: complete console workflows.
- Acceptance: representative academic-office scenarios.

Types

- Functional positive/negative
- Regression
- Performance
- Reliability/fault injection
- Usability

- Security
- Portability/build verification

Entry: stable build, data, environment and baselined requirements/design. Exit: all planned tests executed, no critical defects open, high-priority acceptance criteria verified and RTM complete.

5.1 Security Validation
- Credential storage/source review
- Pre-login authorization attempts
- Malformed/oversized input testing
- File-failure testing
- Failed-login timestamp verification
- Log review for sensitive data

6. Test Environment
7. Test Schedule
8. Test Deliverables
- Test Plan
- Test cases/data
- Test scripts/unit harness
- Execution logs
- Defect reports
- RTM
- Test Summary Report

9. Roles and Responsibilities
10. Risks and Mitigation
11. Assumptions & Dependencies
- Testable build available
- Test credentials/data available
- Reference machine supports dataset
- File-based persistence remains the project scope

12. Suspension & Resumption Criteria
Suspend if the build cannot start, more than 30% of cases are blocked by a common defect, or the environment/data is unusable. Resume after blocking defects are resolved and environment/data are restored.

13. Test Case Management & Traceability
14. Test Metrics & Reporting
- % test cases executed
- Pass/fail rate
- Defect density and aging
- Requirement coverage
- Security coverage
- Performance time/percentile

15. Approvals

Submission Check

- SRS included with editable tables and structured headings.
- SAD included with UML/architecture diagrams.
- STP included with test strategy and detailed test cases.
- Consistent Calibri typography, heading hierarchy, table styling, margins and page numbering applied.
- Table of Contents is based on Word heading styles and can be updated after edits.

Field	Value
Version	1.0
Authors	Samiksha Jayanth; Rohan M; Saksham Bajpayi; Riddhi Mundada
Date	28-09-2026
Status	
Technology	console application, file-based persistence

-

Version	Date	Author	Change Summary
1.0	28-09-2026	Team 4	Architecture and design specification prepared using the supplied SAD template.

-

Role	Name	Signature/Date
Course Coordinator		
Team Representative		

-

Goal/Constraint	Description
Modularity	Separate presentation, controllers, business logic, validation and data access.
Security	Authentication gates record operations; credentials and logs are protected.
Reliability	File I/O errors are handled without corrupting existing records.
Performance	Search/list target: up to 1000 records within 2 seconds on the reference machine.
Constraints	standard library, console UI, local files, single administrator role.

-

Stakeholder	Concern
Administrator	Fast, correct and secure operations
Academic Office	Accurate records and reporting
Developers	Modular maintainable code
QA/Evaluators	Verification and traceability

-

Component	Responsibility	Dependencies
Console UI	Menus, input and output	Authentication, Student Management, Reporting
Authentication	Login, verification, session and failed-login logging	Credential/Log repository
Student Management	CRUD and listing	Validation, Repository
Validation	Mandatory fields, marks and duplicate checks	Student Repository
Reporting/Ranking	Rank list, reports and export	Student data, File Manager
File Manager	Load/save records, credentials, logs and reports	Local files

-

Risk	Mitigation
Write interruption	Safe write handling and return-value checks
Invalid input	Length/type/range validation
Unauthorized access	Authentication gate
Duplicate records	Unique roll-number check
Incorrect calculations	Unit tests with known datasets

-

Requirement	Design Element
FR-001	Authentication + login sequence
FR-002/003/005/006/012	Student Management + Validation + File Manager
FR-004/007	Search/list interfaces
FR-008/009	CGPA and Ranking/Reporting
FR-010/011	File Manager + Report Generator
NFR-001	Search/List performance test
NFR-002/SEC-004	Safe File Manager
SEC-001/002/005	Authentication, credentials and login logging

-

Interface	Operations	Input	Output/Error
IAuth	authenticate(), logout()	username/password	session or failure
IStudentMgmt	add/search/update/delete/list	record or roll number	result/status
IValidation	validateStudent(), validateMarks()	record/marks	valid + reason
IRepository	loadRecords(), saveRecords()	records	success/failure
IReporting	rank/report/export	records/options	report/status

-

Field	Type/Size	Mandatory	Description
roll_no	char[10]	Yes	Unique student roll number
name	char[50]	Yes	Student full name
department	char[20]	Yes	Department/branch
semester	int 1–8	Yes	Current semester
subject_marks[]	float[6], 0–100	Yes	Subject marks
percentage	float	No	System-derived
cgpa	float	No	System-derived
admin_username	char[20]	Yes	Administrator login ID
	char[64]	Yes	Credential representation

-

Field	Value
Version	1.0
Authors	Team 4
Date	28-09-2026
Status	
References	StudentRMS SRS v1.0; StudentRMS SAD v1.0

-

Requirement IDs	Feature
FR-001	Authentication
FR-002/003/012	Add, duplicate check, validation
FR-004	Search/View
FR-005	Update
FR-006	Delete/confirmation
FR-007	List/Sort
FR-008	Percentage/CGPA
FR-009	Rank list
FR-010	Persistence
FR-011	Export
NFR-001..006	Quality attributes
SEC-001..005	Security

-

Item	Environment
Hardware	Reference desktop/laptop
OS	Linux gcc and Windows MinGW
Software	StudentRMS console build
Storage	Isolated local test directory

Tools	Compiler, unit-test harness, shell, text comparison, defect tracker
Data	Valid/invalid records, duplicate IDs, boundary marks, up to 1000 records, credentials

-

Milestone	Date
Test design	28-Sep-2026
Environment setup	28-Sep-2026
Functional execution	29-Sep-2026
Security/reliability	30-Sep-2026
Regression/RTM	01-Oct-2026
Acceptance review	02-Oct-2026

-

Role	Responsibility
QA/Test Lead	Plan, execution coordination, RTM
Test Engineer	Design/execute tests and log defects
Developer	Fix defects and support triage
Product Owner/Team	Review acceptance results

-

Risk	Mitigation
Unstable build	Smoke test before full suite
Bad test data	Controlled datasets
File corruption	Isolated test directory and baseline backups
Late defects	Run high-priority tests early
Platform differences	Cross-platform smoke suite

-

Test ID	Requirement	Scenario	Input/Setup	Expected Result	Type
TC-AUTH-01	FR-001	Valid login	Correct credentials	Session/menu access	Functional
TC-AUTH-02	FR-001,SEC-002	Invalid login	Wrong password	Access denied	Negative/Security
TC-AUTH-03	SEC-005	Failed-login logging	Repeated failures	Timestamped log events	Security
TC-ADD-01	FR-002	Add valid	Unique valid record	Saved successfully	Functional
TC-ADD-02	FR-003	Duplicate roll no.	Existing ID	Rejected	Negative
TC-VAL-01	FR-012	Invalid marks	-1 or 101	Rejected safely	Negative
TC-SEARCH-01	FR-004	Search by roll no.	Existing ID	Correct record	Functional
TC-SEARCH-02	FR-004	Search by name	Existing name	Matching record	Functional
TC-UPD-01	FR-005	Update	Existing record	Updated/persisted	Functional
TC-DEL-01	FR-006	Delete confirmed	Confirm yes	Record removed	Functional
TC-DEL-02	FR-006	Delete cancelled	Confirm no	Record remains	Negative
TC-LIST-01	FR-007	Sort	Multiple records	Correct selected order	System
TC-CGPA-01	FR-008	CGPA	Known marks	Expected value	Unit
TC-RANK-01	FR-009	Rank list	Multiple CGPAs	Correct descending order	System
TC-PERSIST-01	FR-010	Persistence	Save/exit/restart	Records reload	Integration
TC-EXPORT-01	FR-011	Export	Current records	Correct report file	Functional
TC-PERF-01	NFR-001	Performance	Up to 1000 records	Search/list ≤2s target	Performance
TC-REL-01	NFR-002,SEC-004	Write failure	Simulated failure	No corruption; graceful error	Reliability
TC-SEC-01	SEC-001	Credential review	Inspect source/ config	No plaintext embedded	Security
TC-SEC-02	SEC-003	Input robustness	Oversized/ malformed	Rejected without crash	Security
TC-PORT-01	NFR-006	Portability	Linux/Windows builds	Build + smoke pass	Portability

-

Role	Name	Signature/Date
QA Lead		
Developer Lead		
Product Owner/Team		

-

FINAL DOCUMENTATION — CORRECTION & COMPLETION ADDENDUM

StudentRMS — SRS, Software Architecture & Design, and Software Test Plan | Team 4

This addendum is part of the final documentation set and supersedes conflicting placeholder or inaccurate statements in the preceding pages. Existing UML and architecture figures are retained.

A. Corrections to existing pages

Location	Correction
Team profile	Member 3 Saksham Bajpayi: PES1UG24CS404. Member 4 Riddhi Mundada: PES1UG24CS372.
Technology	Use C11 console application with local file-based persistence, not C/C++.
SAD technology/data stores	Current application uses students.dat, login.log and reports.txt. Demo credentials are compiled in src/auth.c; there is no credentials.dat file.
SAD security architecture	Describe the compiled demonstration credential as an accepted academic-scope deviation from original SEC-001 wording.
Student data model	Student has roll_no char[16], name char[50], department char[20], semester int, subject_marks float[6], percentage float and cgpa float. Credential fields are not Student fields.
Document status	Use Final only after actual team approval; until then, identify the set as Final draft — pending team sign-off.

B. SAD — architecture sections

3.1 Goals & Constraints

Goals: separate console interaction from business logic and persistence; validate student data before mutation; keep academic calculations reusable; preserve records across restart; reject corrupt stores safely; support GCC/Linux and MinGW/Windows builds.

Constraints: C11 console application; no network service or web UI; local binary persistence; maximum 1,000 records; six subject marks per student; semester range 1–8; one administrator role; compiled demonstration credential; local report and audit log.

3.2 Stakeholders & Concerns

Stakeholder	Concern	Design response
Department academic office / administrator	Correct, quick record operations	Menu-driven CRUD, search, sorting, CGPA and rank reports.
Student-record data owner	Avoid accidental loss or corruption	Input validation, checked storage results, version/checksum validation and safe replacement.
Developers / maintainers	Clear module boundaries and testability	Separate C modules, public headers, status codes and headless test suites.
QA / course evaluator	Repeatable verification and traceability	Test IDs, traceability CSV, Linux/Windows CI and release verification record.

3.4 Component Descriptions

Component	Responsibility
Console application (src/main.c)	Loads/initializes storage, authenticates, displays the 12-option menu, collects input, calls modules and reports outcomes.
Authentication (src/auth.c)	Maintains process-local session state, checks approved demo credential, logs timestamped events to login.log, and supports logout/session queries.
Student management (src/student_mgmt.c)	Validates fields, semester and marks; checks unique roll numbers; performs add/update/delete with status codes.
Academic computation (src/cgpa.c)	Computes percentage and CGPA from six subject marks.
Search/list (src/search_list.c)	Performs case-insensitive roll/name search, sorting and console display.
Storage (src/storage.c)	Serializes records to a versioned/checksummed local binary store and handles missing, corrupt, capacity and I/O errors.
Reporting (src/report.c)	Writes student listing and CGPA-descending rank list to a text report.

C. SAD — risks, traceability, security and interfaces

3.7 Risks & Mitigations

Risk	Effect	Mitigation / limitation
Interrupted/failed file write	Incomplete or lost records	Temporary-file and backup replacement strategy; checked I/O return values.
Corrupt/incompatible data file	Invalid records may load	Validate header, version, count, checksum and record fields; fail safely.
Invalid/overlong input	Incorrect fields or memory corruption	Bounded line input rejects empty/overlong text; numeric values receive range validation.
Compiler/platform differences	Build/runtime mismatch	Linux GCC and Windows MinGW CI; build.bat is the documented Windows entry point.
Embedded demo credential	Unsuitable for production	Accepted academic-demo-only deviation; synthetic data only.
Incomplete test evidence	Claims cannot be reproduced	Link tests to CI/manual evidence; mark unexecuted scenarios pending.

3.8 Traceability to Requirements

Requirement	Component/design	Verification
FR-001, FR-014, FR-015; SEC-002	Authentication/session functions and startup lockout loop	Authentication tests; logout and lockout CI smoke tests.
FR-002, FR-003, FR-005, FR-006, FR-012	Student Management and Validation modules	Integrated, robustness and Sprint 2 regression tests.
FR-004, FR-007	Search/list module	Search/list suite.
FR-008, FR-009	CGPA and rank/report modules	Known-value calculation and rank-order tests.
FR-010; NFR-002; SEC-004	Storage module	Save/load, restart, corruption and I/O tests.
FR-011	Report module	Report export and content checks.
FR-013; SEC-005	Audit-log menu and authentication logger	CI checks menu output, timestamped events and no password in log.

3.9 Security Architecture

Access control is process-local: the main menu is reached only after successful administrator login, and record-management handlers check authenticated state. Student fields and marks are validated before changes. Storage checks metadata and checksum and returns explicit error codes. Authentication events are timestamped in login.log; passwords are not written to the log. The compiled demo credential is an accepted academic-scope deviation from the original SEC-001 separate-file requirement, not a production control.

4.3 API / Interface Design

Interface	Contract / return behavior
Authentication	auth_login(username,password) returns AUTH_SUCCESS, AUTH_ERR_CREDENTIALS or AUTH_ERR_LOCKED; auth_logout() clears session; auth_is_authenticated() reports session state.
Student management	add_student_record(), update_student_record(), delete_student_record() return MGMT_* status codes; callers report success only on MGMT_SUCCESS.
Search / sort	search_by_roll_no() returns SEARCH_SUCCESS, SEARCH_NOT_FOUND or SEARCH_ERR_PARAM; search_by_name() returns match count or error; sort_students() returns 0 or -1.
Storage	load_students() and save_students() return STORAGE_* status codes for success, missing, corrupt, I/O, capacity or invalid parameters.
Reporting	export_student_reports() returns REPORT_SUCCESS, REPORT_ERR_IO or REPORT_ERR_PARAM.

D. STP — completed test planning sections

3. Features to be Tested

Administrator authentication and session control; student add/update/delete; duplicate and field validation; roll/name search; listing and sorting; percentage/CGPA calculation; rank-list generation; local persistence and restart recovery; report export; audit logging; malformed/overlong input; storage corruption/error handling; Linux/Windows build and performance up to 1,000 records.

4. Features Not to be Tested

University ERP or external API integration; web/mobile GUI; relational database behavior; multi-user concurrency and role hierarchy; OS/compiler internals; production credential management; external authentication providers. These are outside the current console/file-based project scope.

6. Test Environment

Area	Environment
Hardware	Reference desktop/laptop; isolated local test directory; no production data.
Operating systems	Linux and Windows.
Compiler/build	GCC C11; GNU Make on Unix-like systems; MinGW-w64 GCC and build.bat on Windows.
Test tools	Custom C test executables, shell/PowerShell, GitHub Actions, gcov for coverage.
Test data	Synthetic student records, duplicate IDs, boundary marks, malformed/overlong input, missing/corrupt data files.

7. Test Schedule

The executed repository plan records Sprint 1 as 19–23 October 2026 and Sprint 2 as 26–30 October 2026. Feature tests and regression were run during the relevant sprint; release verification and cross-platform CI followed. Do not represent the initial six-sprint estimate as six executed sprints.

9. Roles and Responsibilities

Role	Responsibility
QA / Test Lead	Maintain STP, test execution status, RTM, evidence and defect triage.
Feature owners	Prepare tests for their owned feature, fix defects and provide implementation evidence.
Integration owner	Maintain build workflow, cross-platform integration and release verification.
Team / Product Owner	Review acceptance outcomes, approve scope deviations and provide final release sign-off.

E. STP — risks, traceability, approvals and final evidence

10. Risks and Mitigation

Risk	Mitigation
Unstable build	Run clean compile and smoke tests before full regression.
Bad or sensitive test data	Use synthetic data and isolated working directories; do not commit personal data.
File corruption/interrupted write	Use baseline copies, corruption tests and verify safe failure behavior.
Late defects	Run authentication, CRUD and persistence tests early and repeat regression after fixes.
Platform differences	Run Linux GCC and Windows MinGW CI on the same source revision.
Incomplete evidence	Record exact CI run, environment, tester, date and evidence path; do not mark unexecuted tests passed.

13. Test Case Management & Traceability

Each test case uses a stable ID and maps to one or more SRS requirement IDs. The maintained traceability file is docs/PART3_TEST_CASE_TRACEABILITY.csv. Automated evidence is recorded in GitHub Actions; interactive acceptance evidence is recorded in docs/SPRINT2_RELEASE_VERIFICATION.md. A test is marked PASS only when its expected result is observed. CI run 43254584426 passed Linux GCC, Windows MinGW and GCC coverage, including audit-log viewing, logout, three-attempt lockout and overlong text-input rejection.

15. Approvals

Approval role	Name	Signature/date
QA / Test Lead	To be completed by team	Pending
Developer Lead	To be completed by team	Pending
Product Owner / Team Representative	To be completed by team	Pending
Course Coordinator	To be completed by evaluator	Pending

F. Security disposition and final submission checklist

SEC-001 deviation: the project owner accepts admin / admin123 as demonstration credentials. The original SRS requirement for a separate restricted-access credential file is not technically satisfied; the accepted deviation must be stated in the final submission. The other security controls—pre-login access gating, input validation, safe file handling, timestamped audit logging and no passwords in logs—remain in scope.

Final item	Status
Core application and five test suites	Implemented; CI pass on run 43254584426.
SRS count and security objectives	15 FRs and two security objectives included in the SRS addendum.
SAD missing sections	Completed in this addendum; existing diagrams retained.
STP missing sections	Completed in this addendum; approval signatures remain pending.
Synopsis/SRS SRN and technology corrections	Corrected in this PDF set.
Final team review and release-freeze sign-off	Pending; must be completed by team/evaluator.

Important: This addendum does not fabricate approvals, signatures, or unobserved test results. The team must review the completed documents and sign only after confirming the content.